

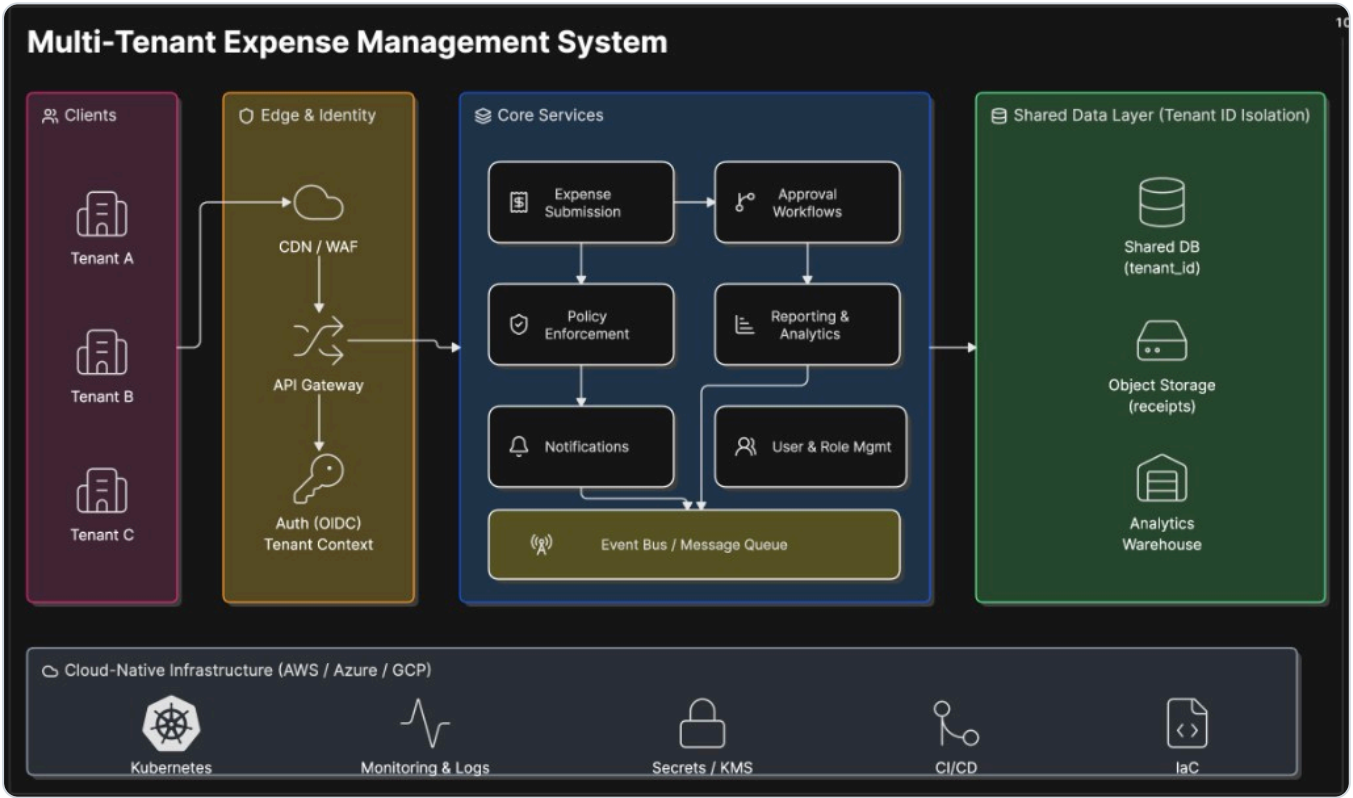
# EMS — Technical Documentation (API Reference & UI Walkthrough)

This document describes the implemented Expense Management System: every API endpoint with request/response samples, with **UI screenshots shown inline next to the flow they belong to**.

- **Base URL:** `http://localhost:4000`
- **Format:** JSON over HTTP
- **Source design:** [level-2-design.md](#) · scope: [level-1.5-scope.md](#)

## Architecture

**Target / production architecture** — multi-tenant clients behind an edge + API gateway, stateless core services on a shared data layer with `tenant_id` isolation, on cloud-native infra:



The as-built prototype implements the same core capabilities as a **modular monolith** (single Express app + PostgreSQL + optional Redis + a pluggable blob store for bill uploads, local-disk today); the gateway, event bus, managed object storage (S3), analytics warehouse and infra layers are the documented production path.

## 1. Conventions

### Authentication

- Obtain a token via `POST /auth/login` or `POST /auth/signup` .
- Send it on every protected request: `Authorization: Bearer <accessToken>` .
- The token encodes `{ id, org_id, role, email }` . The server derives `org_id` from the token — clients never pass it. This is the tenant-isolation guardrail.

## Roles & permissions

| Permission                    | employee | manager | finance | admin |
|-------------------------------|----------|---------|---------|-------|
| expense:create / read own     | ✓        | ✓       | ✓       | ✓     |
| expense:read reportees        | —        | ✓       | —       | ✓     |
| expense:read all              | —        | —       | ✓       | ✓     |
| expense:approve               | —        | ✓       | ✓       | ✓     |
| policy:manage / budget:manage | —        | —       | ✓       | ✓     |
| user:manage                   | —        | —       | —       | ✓     |
| analytics:view                | —        | ✓       | ✓       | ✓     |

## Headers

| Header                                           | When                      | Purpose                        |
|--------------------------------------------------|---------------------------|--------------------------------|
| <code>Authorization: Bearer &lt;token&gt;</code> | all protected routes      | authentication                 |
| <code>Content-Type: application/json</code>      | requests with a body      | parsing                        |
| <code>Idempotency-Key: &lt;unique&gt;</code>     | approve/reject (optional) | prevents double-apply on retry |

## Error envelope

All errors share one shape:

```
{ "error": { "code": "FORBIDDEN", "message": "You do not have permission...", "details": null } }
```

| HTTP | code          | Meaning                                                            |
|------|---------------|--------------------------------------------------------------------|
| 400  | BAD_REQUEST   | Malformed/invalid payload                                          |
| 401  | UNAUTHORIZED  | Missing/invalid token                                              |
| 403  | FORBIDDEN     | Authenticated but not allowed                                      |
| 404  | NOT_FOUND     | Resource not in your tenant                                        |
| 409  | CONFLICT      | Invalid state transition / duplicate                               |
| 422  | UNPROCESSABLE | Valid shape but business rule failed (budget, currency, no policy) |

## Notes on data types

- Postgres `NUMERIC` columns are returned as **strings** (e.g. `"3000.00"` ), preserving precision. Clients should parse as needed.
- Pagination: list endpoints accept `?limit=` (max 100) and `?offset=` .

---

## 2. Auth & Onboarding

---

### POST /auth/signup — create a new organization (tenant onboarding)

Atomically creates an organization, its first **admin** user, a default approval policy and an org budget, then returns tokens (auto-login). Public.

#### Request

```
{
  "orgName": "Initech",
  "baseCurrency": "INR",
  "adminName": "Bill Lumbergh",
  "email": "admin@initech.test",
  "password": "password123"
}
```

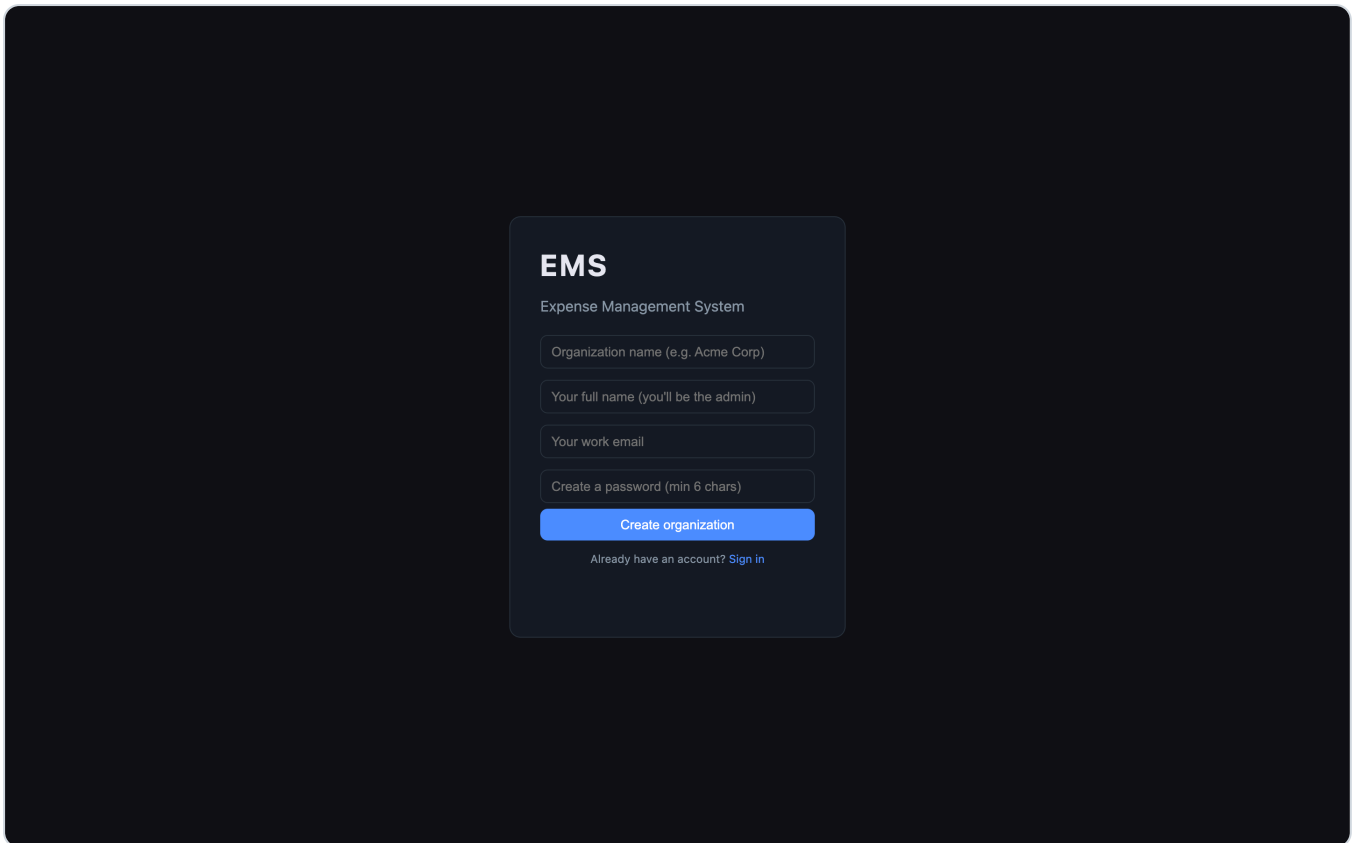
| Field        | Type      | Required | Notes                     |
|--------------|-----------|----------|---------------------------|
| orgName      | string    | yes      | organization name         |
| baseCurrency | string(3) | no       | defaults <code>INR</code> |
| adminName    | string    | yes      | first admin's name        |
| email        | string    | yes      | unique across all tenants |
| password     | string    | yes      | min 6 chars               |

#### Response 201

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": { "id": "uuid", "org_id": "uuid", "name": "Bill Lumbergh", "email": "admin@initech.test",
  "organization": { "id": "uuid", "name": "Initech", "base_currency": "INR" }
}
```

**Errors:** `400` invalid payload · `409` email already exists.

**UI — Sign up (new organization):** "Create an organization" provisions a new tenant + its first admin in one step, with a default policy so it's usable immediately.



**POST** /auth/login

#### Request

```
{ "email": "riya@acme.test", "password": "password123" }
```

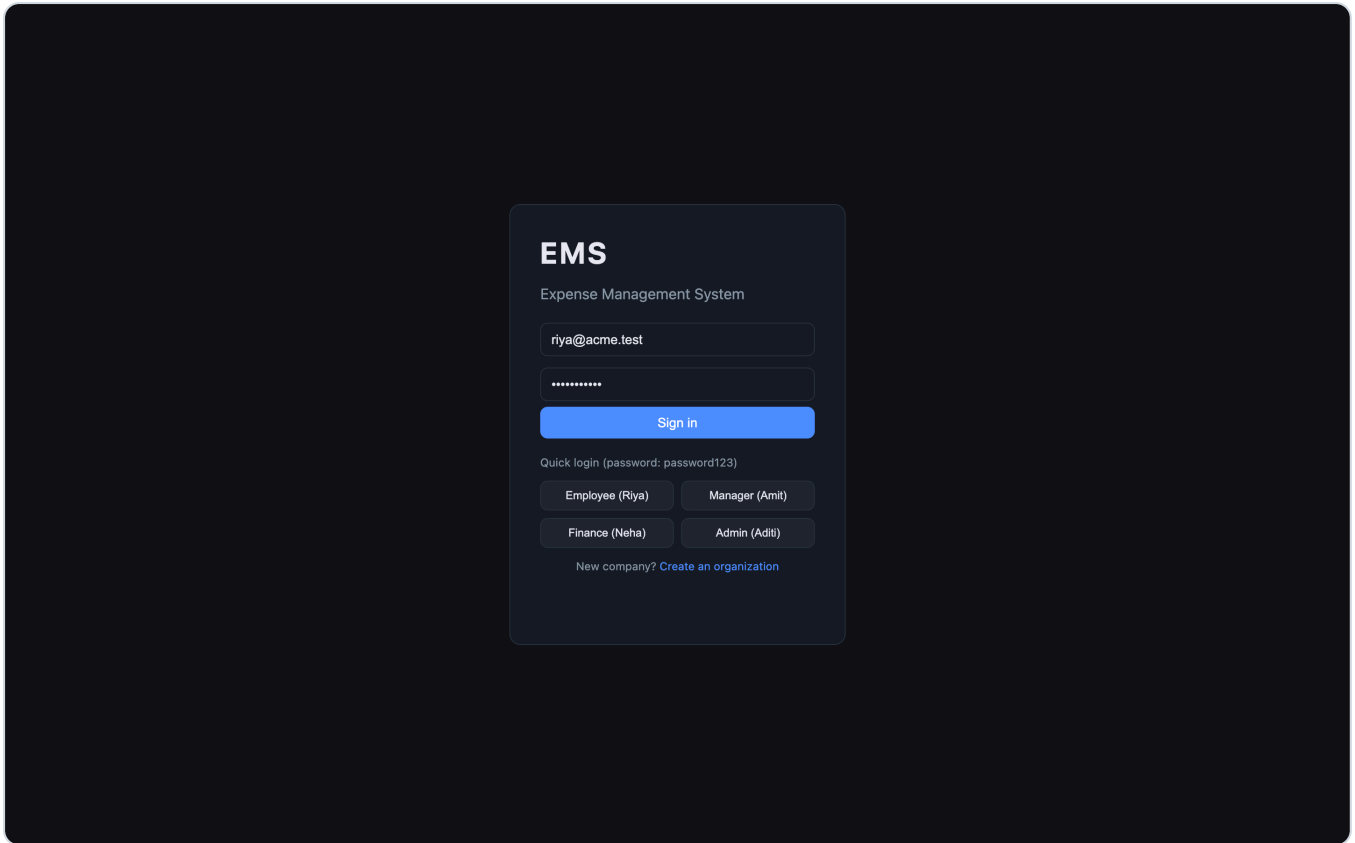
**Response** 200

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": {
    "id": "uuid", "org_id": "uuid", "name": "Riya Sharma",
    "email": "riya@acme.test", "role": "employee",
    "manager_id": "uuid", "is_active": true, "created_at": "2026-..."
  },
  "organization": { "id": "uuid", "name": "Acme Corp", "base_currency": "INR" }
}
```

The `organization` (incl. its `base_currency`) is returned so the UI can format all amounts in the org's currency. `/auth/refresh` returns it too.

**Errors:** 401 invalid email or password.

**UI — Login:** sign in with email/password, or one-click **Quick login** for the seeded roles.



#### POST /auth/refresh — rotate refresh token

Validates an opaque refresh token, **revokes it (single-use rotation)** and issues a fresh access + refresh pair. Replaying a revoked token is treated as reuse and revokes all of the user's sessions.

##### Request

```
{ "refreshToken": "<opaque token from login/signup>" }
```

##### Response 200

```
{ "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXbzI1NiIsImF1dG8iOiB1b250b290dG9u", "refreshToken": "<new opaque token>", "user": { "id": "uuid", "...": "..." } }
```

**Errors:** 401 missing / invalid / expired / reused token.

#### POST /auth/logout

Revokes a refresh token. Idempotent. **Request:** { "refreshToken": "<token>" } · **Response 200** → { "revoked": true }

## 3. System

#### GET /health

**Response 200** → { "status": "ok" }

## GET /metrics

Prometheus text exposition. Public.

```
ems_requests_total 128
ems_errors_total 0
ems_request_latency_ms{quantile="0.5"} 4
ems_request_latency_ms{quantile="0.95"} 21
ems_request_latency_ms{quantile="0.99"} 39
```

## 4. Expenses

### POST /expenses — create a draft

Permission: `expense:create` .

#### Request

```
{ "type": "reimbursement", "category": "travel", "description": "Client visit", "amount": 22000, "cu
```

| Field       | Type   | Required | Notes                                                               |
|-------------|--------|----------|---------------------------------------------------------------------|
| type        | enum   | yes      | <code>reimbursement</code>   <code>company_paid</code>              |
| category    | string | no       | defaults <code>general</code>                                       |
| description | string | no       |                                                                     |
| amount      | number | yes      | > 0, in <code>currency</code>                                       |
| currency    | string | no       | defaults <code>INR</code> ; supported: INR, USD, EUR, GBP, AED, SGD |

The `category` values shown in the UI come from `GET /categories` (admin-managed per org). An expense **cannot be created until the org has an active approval policy** — otherwise there is nothing to route it through.

#### Response 201

```
{
  "id": "uuid", "org_id": "uuid", "requester_id": "uuid",
  "type": "reimbursement", "category": "travel", "description": "Client visit",
  "amount": "22000.00", "currency": "INR", "base_amount": "22000.00", "fx_rate": "1.000000",
  "status": "draft", "policy_snapshot_json": null, "current_level": 0,
  "sla_due_at": null, "created_at": "2026-...", "updated_at": "2026-..."
}
```

**Errors:** 400 invalid · 422 unsupported currency · 422 no active policy configured for the org.

### GET /expenses — list

Query: `scope=mine|reportees|all` (default `mine`), `status`, `type`, `limit`, `offset` .

- `reportees` requires `expense:read:reportees` ; `all` requires `expense:read:all` .

**Response** `200` → array of expense objects (newest first). **Errors:** `403` if scope exceeds your permission.

**UI — Employee "My Expenses":** create reimbursement or company-paid requests and track them with status pills. The amount entered in a foreign currency shows its converted value in the **org base currency** inline (all amounts across the UI are formatted in the org's currency).

| TYPE          | CATEGORY | AMOUNT               | STATUS    |      |
|---------------|----------|----------------------|-----------|------|
| reimbursement | meals    | ₹4,500               | in review | View |
| reimbursement | travel   | ₹16,600 (200.00 USD) | draft     | View |
| reimbursement | travel   | ₹22,000              | approved  | View |
| reimbursement | meals    | ₹3,200               | approved  | View |

**GET** `/expenses/:id` — get one (with approval chain)

**Response** `200`

```
{
  "id": "uuid", "status": "in_review", "current_level": 1, "amount": "22000.00",
  "base_amount": "22000.00", "type": "reimbursement", "category": "travel",
  "steps": [
    { "id": "uuid", "level": 1, "required_role": "manager", "approver_id": "uuid", "status": "approved", "comment": "" },
    { "id": "uuid", "level": 2, "required_role": "finance", "approver_id": "uuid", "status": "pending", "comment": "" }
  ]
}
```

**Errors:** `404` not found / not in your tenant · `403` not allowed to view.

**PATCH** `/expenses/:id` — edit

Only the requester, only while `draft` / `submitted` / `in_review`. If the amount crosses a policy threshold while `in_review`, the approval chain is **rebuilt**. **Request** (any subset): `{ "amount": 20000, "currency": "INR", "category": "...", "description": "..." }` **Response** `200` updated expense. **Errors:** `403`, `409`, `422`.

**DELETE** `/expenses/:id`

Requester only, drafts only. **Response** `200` → `{ "deleted": true }`. **Errors:** `403`, `409`.

### POST /expenses/:id/submit

Moves `draft` → `in_review`, snapshots the policy, builds the `ApprovalStep` chain, and notifies **only the first approver**. Approvals are **sequential**: the next level's approver is notified only after the current level approves; a rejection ends the chain. Before building the chain the engine verifies an **eligible approver exists for every level** (excluding the requester) and fails fast otherwise. **Response 200** → expense with `status:"in_review"`, `current_level:1`. **Errors:** 403 not requester · 409 not a draft · 422 budget exceeded / no active policy / **no eligible approver for a required level**.

Budget-exceeded body:

```
{ "error": { "code": "UNPROCESSABLE", "message": "Budget exceeded for monthly (limit 2000000, already spent 1990000)", "details": { "ok": false, "period": "monthly", "limit": 2000000, "spent": 1990000 } } }
```

### POST /expenses/:id/approve

Permission: `expense:approve`. Must be the assigned approver for the current level; cannot approve your own (approver resolution already routes an expense to a *different* eligible user, so it never lands in the requester's own queue). Approving the last level marks the expense `approved`; otherwise it advances `current_level` and notifies the next approver. Honors `Idempotency-Key`. **Request:** `{ "reason": "looks good" }` (optional). **Response 200** → expense; becomes `approved` after the last level, else stays `in_review` with `current_level` advanced. **Errors:** 403 wrong approver / self-approval · 404 · 409 not awaiting approval.

### POST /expenses/:id/reject

Permission: `expense:approve`. **Reason is mandatory**. Terminates the chain. **Request:** `{ "reason": "not a business expense" }` **Response 200** → expense `status:"rejected"`. **Errors:** 400 no reason · 403 · 404 · 409.

### POST /expenses/:id/withdraw

Requester cancels (`draft` / `submitted` / `in_review`). **Response 200** → `status:"withdrawn"`.

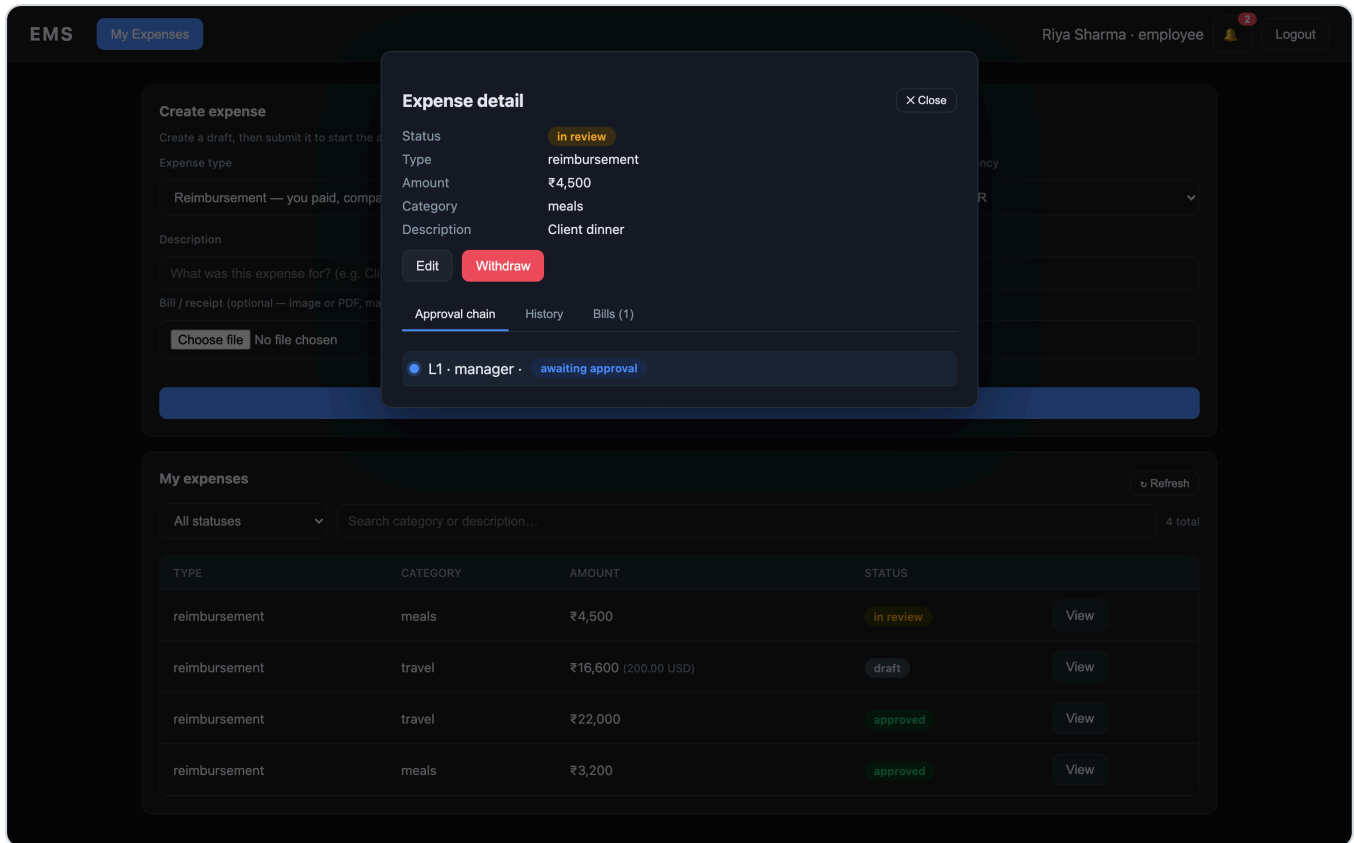
### GET /expenses/:id/history — audit trail

**Response 200**

```
[
  {
    "id": "uuid",
    "actor_id": "uuid",
    "action": "expense.created",
    "entity_type": "expense",
    "entity_id": "uuid",
    "after_json": {
      "action": "expense.submitted",
      "after_json": {
        "status": "in_review",
        "levels": ["manager", "finance"]
      }
    },
    "reason": "ok",
    "created_at": "2026-..."
  },
  {
    "action": "expense.step_approved",
    "reason": "ok",
    "created_at": "2026-..."
  },
  {
    "action": "expense.approved",
    "created_at": "2026-..."
  }
]
```

**UI — Expense detail (chain + audit trail):** each expense shows its computed multi-level **approval chain** (level, role, status) and the immutable **history (audit trail)**. Requester actions (Submit/Withdraw) appear contextually.





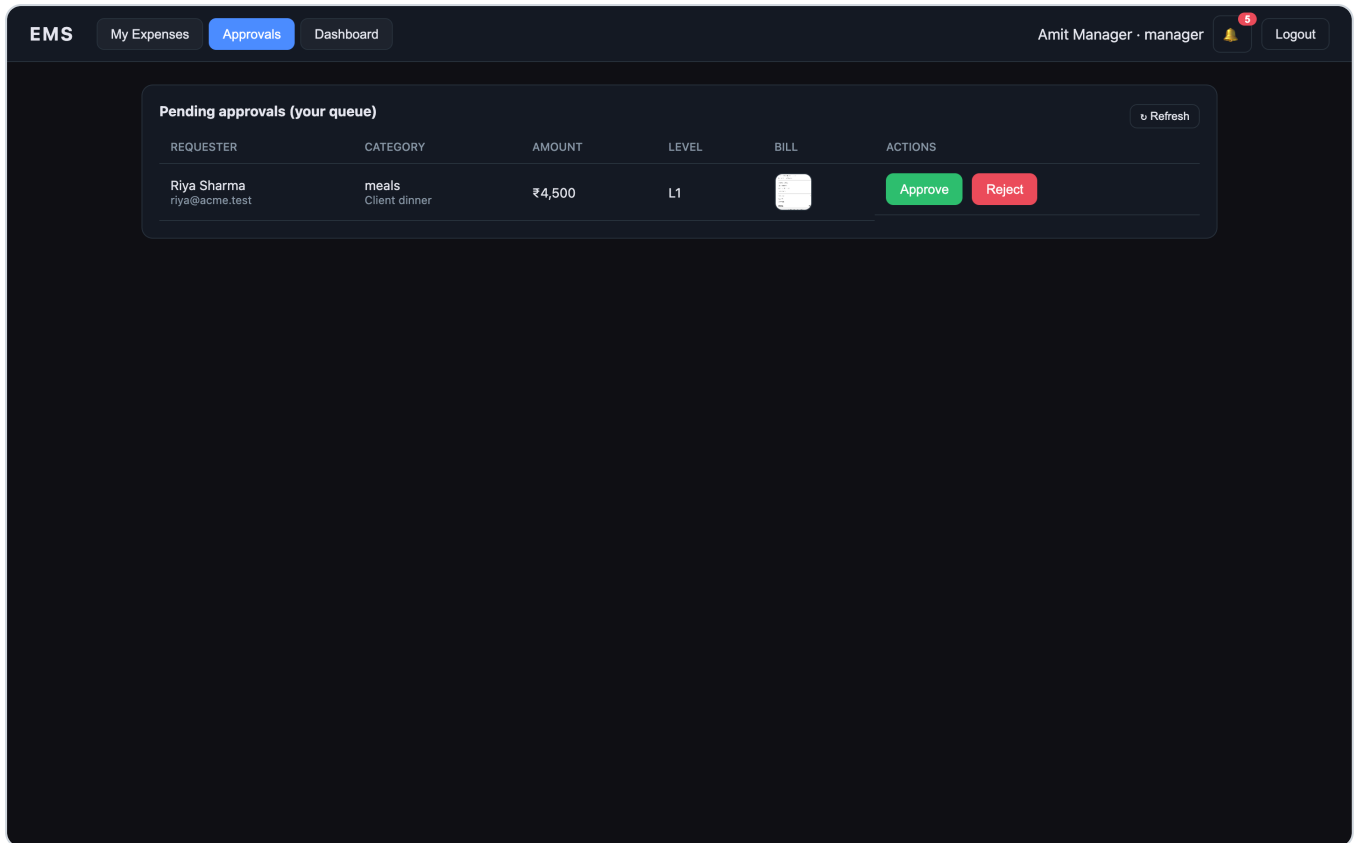
## 5. Approvals

### GET /approvals/pending — my approval queue

Permission: `expense:approve`. Returns steps assigned to you that are pending at the expense's current level. Each row also carries **bill info** so a reviewer can see (and open) the attached receipt without leaving the queue: `attachment_count`, `first_attachment_id`, and `first_attachment_type`. **Response 200**

```
[
  {
    "id": "step-uuid",
    "expense_id": "uuid",
    "level": 1,
    "required_role": "manager",
    "status": "pending",
    "amount": "4500.00",
    "base_amount": "4500.00",
    "currency": "INR",
    "type": "reimbursement",
    "category": "meals",
    "description": "Client dinner",
    "requester_id": "uuid",
    "requester_name": "Riya Sharma",
    "requester_email": "riya@acme.test",
    "expense_status": "in_review",
    "attachment_count": 1,
    "first_attachment_id": "uuid",
    "first_attachment_type": "image/png"
  }
]
```

**UI — Manager approvals queue:** a manager sees only the steps assigned to them at the current level, and can Approve or Reject (reason prompted) directly. The **Bill** column shows the receipt: a single image bill renders as a clickable thumbnail (click to open full size), and PDFs / multiple bills show a  **View** link that opens the expense's Bills tab. An **assigned approver can always view the expense and its bills** even when the requester is not one of their direct reportees (the same visibility rule the bill download endpoint enforces).



## 6. Attachments (bill uploads)

Attach a **bill** — a receipt or invoice — to an expense. Files are uploaded to the API as multipart form data, the **bytes are stored through a `BlobStorage` abstraction** (default `LocalDiskStorage` ; production swaps in S3 with the same interface), and the **metadata** (filename, content-type, size, uploader) is written to Postgres. Allowed types: **PNG, JPEG, WebP, GIF, or PDF**, up to **5 MB**.

### `POST /expenses/:id/attachments` — upload a bill

`multipart/form-data` with a single field `file` . Only the **requester** can attach a bill, and only while the expense is still editable ( `draft` / `submitted` / `in_review` ).

```
curl -X POST localhost:4000/expenses/$EID/attachments \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@/path/to/receipt.png;type=image/png"
```

### Response `201`

```
{
  "id": "uuid", "org_id": "uuid", "expense_id": "uuid",
  "s3_key": "ems-attachments/<org>/<expense>/<uuid>-receipt.png",
  "filename": "receipt.png", "content_type": "image/png",
  "size": 45512, "uploaded_by": "uuid", "uploaded_at": "2026-..."
}
```

**Errors:** `400` no file · `403` not the requester · `409` expense no longer editable · `422` unsupported file type / file too large.

## GET /expenses/:id/attachments — list bills

Anyone allowed to view the expense (requester, their manager, finance/admin). **Response** **200** → array of attachment metadata (no bytes).

## GET /attachments/:id — download / preview a bill

Streams the stored bytes with the original **Content-Type** and an inline **Content-Disposition**. **Every download is authenticated and tenant-scoped** — bills are never served as public static files. Same visibility rules as the expense. **Errors:** **403** not allowed · **404** not found / other tenant.

**UI — attach a bill at creation:** the bill picker is on the **create expense** form, so a receipt/invoice can be attached up front (the UI creates the draft and uploads the bill in one step). It can also be added later from the detail **Bills** tab.

EMS

My Expenses

Riya Sharma · employee

2

Logout

Create expense

Create a draft, then submit it to start the approval flow.

Expense type

Reimbursement — you paid, company

Category

Accommodation

Amount

1850

Currency

INR

Description

Client dinner — receipt attached

Bill / receipt (optional — image or PDF, max 5MB)

Choose file

ems-sample-receipt.png

Create draft

My expenses

Refresh

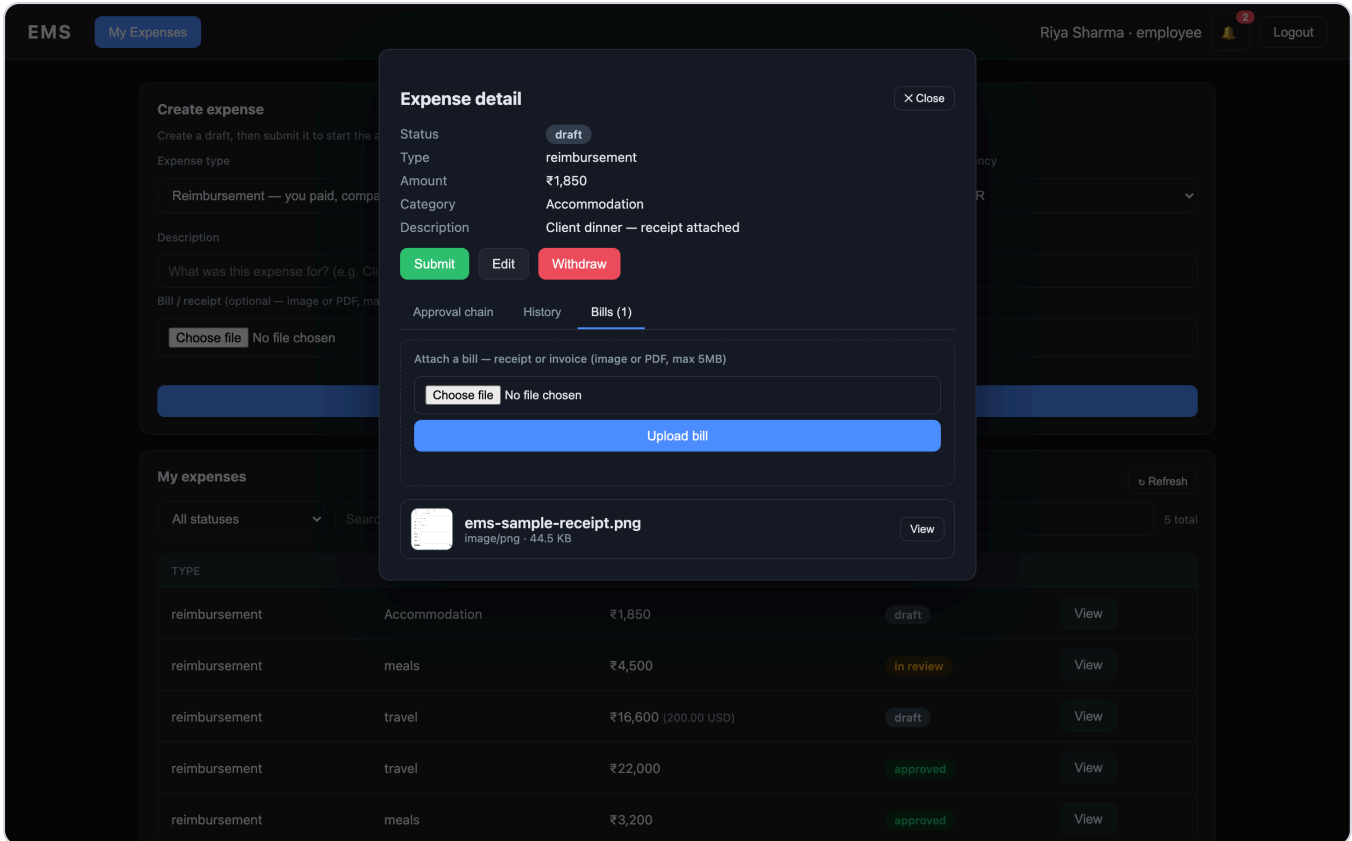
All statuses

Search category or description...

4 total

| TYPE          | CATEGORY | AMOUNT               | STATUS    |                 |
|---------------|----------|----------------------|-----------|-----------------|
| reimbursement | meals    | ₹4,500               | in review | <div>View</div> |
| reimbursement | travel   | ₹16,600 (200.00 USD) | draft     | <div>View</div> |
| reimbursement | travel   | ₹22,000              | approved  | <div>View</div> |
| reimbursement | meals    | ₹3,200               | approved  | <div>View</div> |

**UI — Bills tab:** uploaded bills show as thumbnails (images) and open in a new tab via **View**. Reviewers see the same bills while approving.



### POST /attachments/presign — mock S3 presigned URL (documented prod path)

The alternative to API-proxied uploads is a **direct-to-bucket** PUT using a pre-signed URL. This endpoint returns a *mock* URL to document that flow; no real S3 call is made in the prototype. **Request:** `{ "filename": "bill.png", "contentType": "image/png" }` **Response 200**

```
{
  "key": "ems-attachments/<org>/<uuid>-bill.png",
  "uploadUrl": "https://mock-s3.local/...?X-Amz-Signature=mock&expires=900",
  "method": "PUT", "expiresInSeconds": 900,
  "note": "MOCK presigned URL (no real S3 in prototype)"
}
```

**Errors:** 400 missing filename/contentType.

## 7. Policies

### GET /policies

Permission: analytics:view . **Response 200** → array:

```
[{ "id":"uuid","name":"Default approval policy","tolerance_percent":"10.00","active":true,"version":1,
  "rules_json":{ "currency":"INR","rules":[
    { "min":0,"max":5000,"levels":["manager"]},
    { "min":5001,"max":50000,"levels":["manager","finance"]},
    { "min":50001,"max":null,"levels":["manager","finance","admin"]}]}]}
```

**Exactly one policy is active per org.** The approval engine always uses the single active policy, so the UI never shows several "active" rows competing to route an expense.

## POST /policies — create

Permission: `policy:manage` . Creating a policy as `active` (the default) **deactivates every other policy in the org in the same transaction**. Policy names must be unique within the org. **Request**

```
{ "name": "Travel policy", "tolerancePercent": 10,
  "rulesJson": { "currency": "INR", "rules": [ { "min": 0, "max": 10000, "levels": ["manager"] } ] } }
```

**Response** `201` policy object. **Errors:** `400` invalid · `409` duplicate policy name · `422` a rule's `max < min` .

## PATCH /policies/:id — update (bumps version)

Any subset of `name` , `tolerancePercent` , `active` , `rulesJson` . Setting `active:true` activates this policy and **deactivates all others** atomically. **Errors:** `404` if missing · `409` duplicate name.

## DELETE /policies/:id

**Response** `200` → `{ "deleted": true }` . **Errors:** `404` if missing · `400` the **active policy can't be deleted** (activate another one first).

**UI — Policies: a visual rule builder** (amount band → ordered approver chain, no JSON to hand-write) plus the tolerance percentage. The list shows which policy is active, lets you activate/deactivate with one click, and delete inactive ones. Admins also manage the org's **expense categories** here (the list that feeds the expense-form dropdown).

The screenshot shows the 'Create approval policy' form in the EMS application. The top navigation bar includes 'EMS', 'My Expenses', 'Approvals', 'Dashboard', and 'Policies' (highlighted). The user is 'Neha Finance' in the 'finance' role, with a notification badge showing '2' and a 'Logout' button.

The form is titled 'Create approval policy' and includes a sub-header: 'Only one policy is active at a time — the active policy decides who approves each expense based on its amount. Creating a policy makes it the active one.'

The form has two main sections: 'Policy name' and 'Tolerance % (allowed variance for company-paid amounts)'. The 'Policy name' field contains 'e.g. Standard approval policy'. The 'Tolerance %' field contains '0'.

Below these fields is a section titled 'Approval bands — for each amount range, tick who must approve'. It contains three rows of input fields for 'From (₹)' and 'To (₹, blank = no limit)'. The first row has '0' and '5000'. The second row has '5001' and '50000'. The third row has '50001' and '∞'.

Each row has a 'Must be approved by (in this order)' section with checkboxes for 'manager', 'finance', and 'admin'. The first row has 'manager' checked. The second row has 'manager' and 'finance' checked. The third row has 'manager', 'finance', and 'admin' checked.

At the bottom of the form are two buttons: '+ Add band' and 'Use standard template'.

## 7b. Categories

---

Per-org expense categories that populate the category dropdown on the expense form. Seeded with sensible defaults on signup; admins can add/remove more.

### GET /categories

Permission: any authenticated user. **Response** 200

```
[{ "id":"uuid","org_id":"uuid","name":"travel","active":true,"created_at":"2026-..." }]
```

### POST /categories — add a category

Permission: policy:manage . **Request:** { "name": "software" } **Response** 201 category object. **Errors:** 400 invalid · 409 duplicate name.

### DELETE /categories/:id — soft-deactivate

Permission: policy:manage . Marks the category inactive (history on existing expenses is preserved). **Response** 200 → { "deleted": true } . 404 if missing.

---

## 8. Budgets

Budgets are enforced **per person**: a user's own approved + in-review spend in the window must stay within their limit. A `scope:"org" budget ( user_id null)` acts as the **fallback** for users without a personal limit. Amounts are in the org base currency.

**GET /budgets (perm analytics:view ) → array of budgets.**

**POST /budgets (perm budget:manage )**

**Request:** { "userId": "uuid|null", "scope": "user|org", "period": "daily|monthly", "limitAmount": 50000, "currency": "INR" } **Upsert:** setting a budget for an existing (org, user, scope, period) slot **updates the same row** instead of inserting a duplicate, so re-setting a person's monthly limit never creates ambiguous rows. **Response** 201 budget object. **Errors:** 400 invalid (a scope:"user" budget requires userId ) · 403 without budget:manage .

**GET /budgets/spend (perm budget:manage ) — per-user current-month spend**

**Response** 200 → { "<userId>": 25200, "<userId2>": 0 } (base currency). Powers the admin Budgets screen.

**GET /budgets/utilization — current user's usage**

**Response** 200 → { "monthly": { "limit": 2000000, "spent": 25200, "pct": 1 } }

**UI — Admin Budgets:** the **Budgets** tab lists every active user with their current-month spend, effective monthly limit (personal or org default), and utilization %, plus an inline field to set each person's monthly limit. Over-limit users are flagged. Submitting an expense that would push a user over their limit is blocked with a 422 .

---

## 9. Users

**GET** `/users` (perm `user:manage`) → array (password hashes stripped).

**POST** `/users` (perm `user:manage`)

**Request:** `{ "name": "New", "email": "new@acme.test", "password": "secret1", "role": "employee", "managerId": "uuid|null" }` **Response** `201` user (no hash). **Errors:** `400` invalid · `409` duplicate email.

**PATCH** `/users/:id` (perm `user:manage`)

Any subset of `name`, `role`, `managerId`, `isActive`. Setting `isActive:false` **deactivates** the user — they can no longer log in and are skipped by approver resolution; `isActive:true` reactivates them. **Guards:** you cannot deactivate **your own account**, nor the **last active admin** in the org (→ `400` / `409`). `404` if missing.

**UI — Admin Users:** admins add employees/managers/finance to their org (with optional manager assignment) and view the org roster with active/inactive status. Each row has inline **role** and **reports-to (manager)** dropdowns, so a user's manager can be assigned, changed, or cleared after creation, alongside deactivate/reactivate.

## 10. Notifications

**GET** `/notifications` → my notifications (newest 100).

```
[{"id":"uuid","type":"approval_requested","payload_json":{"expenseId":"uuid","level":1},"read":false}]
```

`POST /notifications/:id/read` → `{ "read": true }`. `404` if not yours.

---

## 11. Analytics / Observability

---

All require `analytics:view`. Summary is Redis/memory-cached (short TTL).

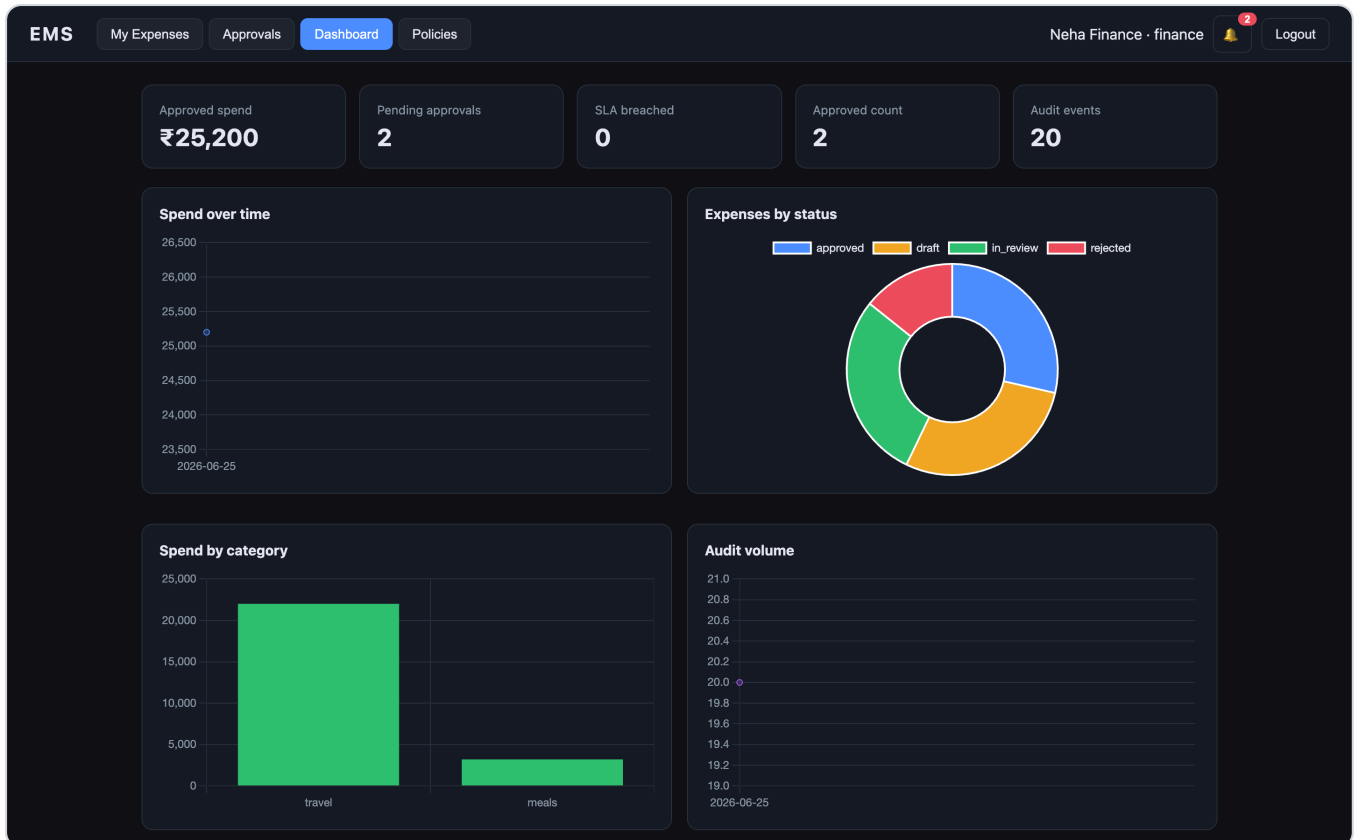
| Endpoint                                 | Response                                                                                                  |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>GET /analytics/summary</code>      | <code>{ approvedSpend, approvedCount, pendingApprovals, slaBreached, auditEvents, byStatus:{...} }</code> |
| <code>GET /analytics/spend</code>        | <code>[{ "day":"2026-06-24", "total": 25200 }]</code>                                                     |
| <code>GET /analytics/by-status</code>    | <code>[{ "status":"approved", "count": 2 }]</code>                                                        |
| <code>GET /analytics/by-category</code>  | <code>[{ "category":"travel", "total": 22000 }]</code>                                                    |
| <code>GET /analytics/audit-volume</code> | <code>[{ "day":"2026-06-24", "count": 16 }]</code>                                                        |

Example `GET /analytics/summary` :

```
{ "approvedSpend": 25200, "approvedCount": 2, "pendingApprovals": 2, "slaBreached": 0,
  "auditEvents": 16, "byStatus": { "approved": 2, "draft": 1, "in_review": 2, "rejected": 1 } }
```

**UI — Finance/Admin observability dashboard:** stat cards (approved spend, pending, SLA breached, audit events) plus charts — spend over time, expenses by status, spend by category, audit volume. Data is aggregated from Postgres and cached.





## 12. Security, isolation & concurrency

**Authentication.** Access tokens are short-lived JWTs ( { id, org\_id, role, email } ). Refresh tokens are **opaque random strings stored only as SHA-256 hashes** in `refresh_tokens` . `/auth/refresh` does **single-use rotation** (old token revoked, new one issued, chained via `replaced_by` ); replaying a revoked token is treated as theft and revokes the user's whole session family. `/auth/logout` revokes a token. (Code: `src/auth/refreshToken.ts` .)

**Tenant isolation.** `org_id` is read from the verified token (never the client) and applied to every query — the core isolation guardrail. This is app-layer isolation; one missing predicate would leak across tenants. The production hardening is **Postgres Row-Level Security**:

```
ALTER TABLE expense_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON expense_requests
  USING (org_id = current_setting('app.org_id')::uuid);
-- set per transaction: SET LOCAL app.org_id = '<org from token>';
```

It's documented rather than enabled here because RLS interacts with the connection pool and the migration/seed paths; the app-layer guard plus this plan is the deliberate tradeoff.

**Concurrency (no double-apply).** Every state transition (submit/edit/approve/reject/withdraw) opens a transaction and takes an exclusive row lock via `SELECT ... FOR UPDATE ( getByIdForUpdate )` . Concurrent approvals on the same expense are serialized by Postgres: the loser blocks, re-reads the committed state, and is rejected by the status/level guards. Idempotency keys are orthogonal — they dedupe identical retries, the row lock handles distinct concurrent requests.

## 13. End-to-end flow (recap)

---

```
signup/login → create expense (draft) → submit
  → engine reads policy, snapshots it, builds ApprovalStep chain by amount
  → notify approver(s)
manager approve → (advance level) → finance approve → ... → APPROVED
                |
                reject (reason) → REJECTED
every transition → immutable audit_log entry (in the same DB transaction)
finance/admin → dashboard reads aggregates for live observability
```